

COMP1850 Programming

School of Computer Science, University of Leeds

Worksheet 1.5

This worksheet contains a combination of formative activities and activities that contribute towards your portfolio.

- Portfolio activities are indicated in the title of the task.
- Activities marked by (*) are advanced activities and may take more time to complete.

Expectations:

1. Timeliness: You should complete all the tasks in the order provided and submit your portfolio task to Gradescope before the deadline.
2. Presentation: You should present all your work clearly and concisely following any additional guidance provided by the module staff in the module handbook.
3. Integrity: You are responsible that the evidence you submit as part of your portfolio evidence is **entirely your own work**. You can find out more about Academic integrity on the Skills@library website. All work you submit for assessment is subject to the academic integrity policy.

Feedback: Feedback on formative activities will be provided via Lab classes and tutorials. Feedback on evidence submitted as part of the portfolio will be available on Gradescope.
Support opportunities: Support with the activity sheet is available in the Lab classes and tutorials. Individual support is available via office hours.
Expected complete date: **Friday 6th November**

Portfolio tasks should be entirely your own work. AI tools (such as Microsoft Copilot) can be used in an assistive role. You are permitted to use AI tools for specific defined processes within the assessment.

Within this assessment you **may** use Generative AI to:

- help you start to understand more complex ideas by providing accessible summaries.
- test your knowledge against a piece of content.
- help identify and correct spelling mistakes and grammatical errors in your work.
- provide feedback and advice on your overall coding style.

You must **not** use Generative AI to:

- produce written content (code, comments or any other content) which you then submit as your work, regardless of whether you make changes to it.
- rewrite or make changes to any of your work.

Learning Outcomes:

- Develop classes with constructors and methods

Portfolio Task

You must create two classes for this week's portfolio task. Both will be tested on Gradescope. All the tests must pass to achieve a passing grade for the task.

Rectangle Class

Edit `rect.py` and complete the constructor so that it checks the type and value of the `width` and `height` constructor parameters. Your constructor should raise a `TypeError` exception if either parameter is not of type `int`. It should raise a `ValueError` exception if either parameter is less than or equal to 0.

After finishing and testing the constructor, implement the following methods:

- `is_square()`, which should return `True` if the rectangle is actually a square, `False` otherwise
- `area()`, which should return the area of the rectangle
- `perimeter()`, which should return the perimeter of the rectangle

Give each of these methods a suitable docstring. If you don't provide a docstring for each of them, the tests will fail.

Do not change the name of the file, the class, or any of these methods, or the tests will fail.

Do not add anything else to `rect.py`.

Tip: Do not upload a solution without first checking that it actually works! You can do this by writing a small demo program, **in a separate file**. This program should create a `Rectangle` object and then call its methods.

Person Class

Edit `person.py`. Implement a constructor in the `Person` class that creates and initializes the fields `name` and `birth`. ~The former represents a person's name, the latter represents their date of birth. Use these exact names and remember to use the '`self.`' prefix.

Your constructor should raise a `TypeError` if the value provided for `name` is not a string. It should raise a `ValueError` if the value provided for `name` is an empty string.

The constructor should also raise a `TypeError` if the value provided for `birth` is not a `date` object.

After finishing and testing your constructor, give your `Person` class a method named `initials()`. This should return the uppercased initial letters of each part of a person's name, joined together into a single string. For example, if the person's name is Alan John Smith, the method should return `"AJS"`.

Finally, give your `Person` class a method `is_birthday()`. This should return `True` if today is the person's birthday, `False` otherwise (see below for some hints on how to do this).

Give the constructor and each of the methods a suitable docstring. If you don't provide docstrings, the tests will fail.

Do not change the name of the file, the class, or any of these methods, or the tests will fail.

Do not add anything else to `person.py`.

Tip 1: Do not upload a solution without first checking that it actually works! You can do this by writing a small demo program, **in a separate file**. This program should create a `Person` object and then call its methods.

Tip 2: You will find the documentation for Python's `datetime` module to be useful, particularly the section dealing with the date class. This class has a method `today()` which you can call to get today's date as a `date` object.